



**Universidade Estadual de Campinas
Faculdade de Tecnologia**

André Luiz Clemente de Oliveira - 269943

Projeto Prático – Rotaciona Matriz
TT304 – Sistemas Operacionais

Limeira, 2026



1. Introdução

Este relatório descreve o desenvolvimento do projeto prático da disciplina TT304, Sistemas Operacionais, cujo objetivo é a implementação de um programa em linguagem C que utilize a biblioteca *POSIX Threads* (*pthread*) para rotacionar uma matriz quadrada $N \times N$ em 90° no sentido horário, fazendo uso eficiente dos recursos de processamento paralelo disponíveis.

O projeto explora um conceito fundamental de sistemas operacionais, criação e sincronização de threads. Além disso, foram aplicadas técnicas de otimização de desempenho, como alocação contínua de memória para melhor localidade de cache, variáveis em registradores e distribuição balanceada de carga entre threads.

2. Resolução do Problema

Dada uma matriz quadrada $N \times N$ de inteiros (positivos ou negativos), o programa deve rotacioná-la 90° no sentido horário. A rotação é realizada in-place, ou seja, sem necessidade de uma segunda matriz auxiliar, em dois passos:

Passo 1 – Transposta:

Cada elemento $\text{mat}[i][j]$ é trocado com $\text{mat}[j][i]$, processando apenas o triângulo superior ($j > i$) para evitar que a mesma posição seja trocada duas vezes.

Passo 2 – Espelhamento horizontal:

Cada linha da matriz transposta é espelhada horizontalmente: $\text{mat}[i][j] \leftrightarrow \text{mat}[i][N-1-j]$, percorrendo apenas a metade esquerda da linha ($j < N/2$).

Exemplo para matriz 4×4 :

Original:	Transposta:	Rotacionada 90° :
7 2 -3 1	7 42 -12 26	26 -12 42 7
42 53 6 4	2 53 17 1	1 17 53 2
-12 17 22 97	-3 6 22 19	19 22 6 -3
26 1 19 43	1 4 97 43	43 97 4 1

Esta abordagem garante que não tenha condição de corrida entre as threads, sem utilizar uma matriz auxiliar que iria duplicar a quantidade de memória necessária para realizar a rotação.

3. Instruções de Compilação

3.1 Requisitos



- Sistema operacional Linux
- Compilador GCC com suporte a C99
- Biblioteca *POSIX* Threads (incluída no GCC com a flag `-pthread`)
- Utilitário `make`

3.2 Compilação com Makefile

O repositório inclui um Makefile que automatiza a compilação. Para compilar:

```
make                # compila o rotacionaMat
make geraMatriz     # compila o gerador de matrizes de teste
make test           # executa todos os experimentos automaticamente
make clean          # remove binários e arquivos gerados
```

3.3 Compilação manual

Caso prefira compilar manualmente, utilize os seguintes comandos:

```
gcc -Wall -Wextra -O2 -std=c99 -pthread -o rotacionaMat
rotacionaMat.c
gcc -Wall -Wextra -O2 -std=c99 -o geraMatriz geraMatriz.c
```

3.4 Execução

O programa recebe quatro parâmetros pela linha de comando:

```
./rotacionaMat <N> <T> <ArqEntrada> <ArqSaida>
```

- `N`: dimensão da matriz ($N \times N$)
- `T`: número de threads de processamento
- `ArqEntrada`: arquivo texto com a matriz original
- `ArqSaida`: arquivo de saída com a matriz rotacionada (`.rot`)

Exemplo de execução:

```
./geraMatriz 1000 matriz.dat
./rotacionaMat 1000 4 matriz.dat matriz.rot
```



4. Lógica do Programa

4.1 Alocação de Memória

A matriz é alocada em um único bloco contíguo de memória (um único malloc para os dados e outro para o vetor de ponteiros de linha), garantindo localidade de cache e atendendo ao requisito do projeto de alocação em etapa única.

4.2 Distribuição de Trabalho entre Threads

O trabalho é distribuído de forma intercalada entre as T threads. Cada thread de id t processa as linhas t , $t+T$, $t+2T$, etc. Essa distribuição resolve o desbalanceamento natural do triângulo superior, onde as primeiras linhas têm muito mais elementos que as últimas: cada thread recebe tanto linhas pesadas quanto linhas leves, equilibrando a carga.

4.3 Sincronização com Barreira

Uma barreira POSIX (*pthread_barrier_t*) é utilizada entre os dois passos, todas as threads devem concluir a transposição antes que qualquer uma inicie o espelhamento. Isso é necessário porque a transposição de uma linha escreve em posições de outras linhas, sem a barreira, uma thread poderia espelhar uma linha que outra thread ainda não terminou de transpor, corrompendo os dados. Além disso, foi utilizado o *pthread_join* de forma análoga a uma barreira, fazendo o código parar e esperar as threads terminarem o espelhamento.

4.4 Localidade de Cache

A transposta percorre as linhas de forma intercalada entre threads, e dentro de cada linha processa as colunas sequencialmente a partir de $i+1$ (triângulo superior). Esse padrão de acesso sequencial às colunas favorece o *prefetch* de cache, já que os elementos $mat[i][j]$ estão contíguos na memória (alocação em bloco único). A alocação contínua da matriz garante que linhas inteiras estejam próximas na memória, reduzindo *cache misses*.

4.5 Medição de Tempo

O tempo de cada thread é medido individualmente com `clock_gettime(CLOCK_MONOTONIC)`, com precisão de nanossegundos. São realizadas duas medições de intervalo por thread: o tempo da transposição e o tempo do espelhamento. Utilizando esses dois tempos, foi possível calcular o tempo total de rotação.



5. Resultados dos Experimentos

Os experimentos foram realizados com matrizes de 1000×1000 inteiros, variando o número de threads de processamento entre 1, 2, 4 e 8. Os tempos de I/O (leitura e gravação de arquivos) não são contabilizados, apenas o tempo de processamento (transposta e espelhamento) é medido.

5.1 Tabela de Tempos de Execução

Threads (T)	Tempo Total Teste 1	Tempo Total Teste 2	Tempo Total Teste 3	Tempo Total Teste 4
1	0.001256307	0.001241293	0.001161483	0.001267170
2	0.000813061	0.000951057	0.001088906	0.000996100
4	0.001251447	0.001349527	0.001076349	0.001015483
8	0.001048962	0.001166859	0.001091813	0.001172557

Tabela 1 – Tempos de execução para matriz 1000×1000 com diferentes números de threads.

5.2 Análise dos Resultados

Os resultados mostram que o maior ganho de desempenho ocorreu na transição de $T=1$ para $T=2$ threads. A partir de $T=4$, os tempos se estabilizaram ou até pioraram, indicando que o gargalo deixou de ser processamento e passou a ser o acesso à memória, criação de threads e outras operações secundárias.

Os principais fatores que podem explicar esse comportamento são:

- **Saturação de memória (*memory-bound*):** a rotação de matrizes é uma operação intensiva em acessos à memória. Com mais threads, o barramento de memória satura antes que os núcleos da CPU.
- **Overhead de *context switch*:** com T maior que o número de núcleos físicos disponíveis, o sistema operacional divide o tempo de CPU entre threads, adicionando overhead de escalonamento.
- **Overhead da barreira:** com mais threads, a probabilidade de uma thread atrasar todas as outras na barreira aumenta, reduzindo o ganho do paralelismo.

- Custo de criação de threads: para tarefas muito rápidas, o custo de criar e destruir threads pode superar o ganho do paralelismo.

6. Conclusão

O projeto demonstrou que o uso de múltiplas threads com a biblioteca *POSIX Threads* pode acelerar significativamente as operações matriciais. A rotação in-place eliminou a necessidade de matriz auxiliar, reduzindo o uso de memória à metade. As técnicas de armazenamento contínuo e distribuição intercalada de linhas contribuíram para um código eficiente e bem balanceado.

Os experimentos confirmaram que o ganho de desempenho com múltiplas threads não é linear e depende diretamente do hardware disponível. Para a tarefa de rotação de matrizes 1000×1000 , o ponto ótimo foi $T=2$ threads, enquanto $T=4$ e $T=8$ apresentaram aumento e estabilização no tempo.

Por fim, a técnica aplicada provavelmente demonstraria mais eficiência em matrizes maiores (5000×5000 ou 10000×10000), onde o custo de criação de threads se torna desprezível em relação ao trabalho realizado, permitindo observar ganhos mais expressivos com maior número de threads.